Chapter 12. Training for Ranking

Typical ML tasks usually predict a single outcome, such as the probability of being in a positive class for classification tasks, or an expected value for regression tasks. Ranking, on the other hand, provides a relative ordering of sets of items. This kind of task is typical of search results or recommendations, where the order of items presented is important. In these kinds of problems, the score of an item usually isn't shown to the user directly but rather is presented—maybe implicitly—with the ordinal rank of the item: the item at the top of the list is numbered lower than the next item.

This chapter presents various kinds of loss functions that ML algorithms can use during training. These scores should estimate list orderings such that when compared to one another, they result in sets that are ordered more closely to the relevance ordering observed in a training dataset. Here we will focus on introducing the concepts and computations, which you'll put to work in the next chapter.

Where Does Ranking Fit in Recommender Systems?

Before we dive into the details of loss functions for ranking, we should talk about where ranking fits into the larger scheme of recommender systems as a whole. Typical large-scale recommenders have a retrieval phase, in which a cheap function is used to gather a decent number of candidate items into a candidate set. Usually, this retrieval phase is only item based. For example, the candidate set might include items related to recently consumed or liked items by a user. Or if freshness is important, such as for news data, the set might include the newest popular and rele-

vant items for the user. After items are gathered into a candidate set, we apply ranking to its items.

Also, since the candidate set is usually much smaller than the entire corpus of items, we can use more expensive models and auxiliary features to help the ranking. These features could be user features or context features. User features could help in determining the items' usefulness to the user, such as the average embedding of recently consumed items. Context features could indicate details about the current session, such as time of day or recent queries that a user has typed—a feature that differentiates the current session from others and helps in determining relevant items. Finally, we have the representation of the items themselves, which can be anything from content features to learned embeddings that represent the item.

The user, context, and item features are then concatenated into one feature vector that we will use to represent the item; we then score all the candidates at once and order them. The rank ordered set might then have extra filtering applied to it for business logic, such as removing near duplicates or making the ranked set more diverse in the kinds of items displayed.

In the following examples, we will assume that the items can all be represented by a concatenated feature vector of user, context, and item features and that the model could be as simple as a linear model with a weight vector `W` that is dotted with the item vector to obtain a score for sorting the items. These models can be generalized to deep neural networks, but the final layer output is still going to be a scalar used to sort the items.

Now that we have set the context for ranking, let's consider ways we might rank a set of items represented by vectors.

Learning to Rank

Learning to rank (LTR) is the name for the kind of models that score an ordered list of items according to their relevancy or importance. This

technique is how we go from the potentially raw output of retrieval to a sorted list of items based on their relevance.

LTR problems have three main types:

Pointwise

The model treats individual documents in isolation and assigns them a score or rank. The task becomes a regression or classification problem.

Pairwise

The model considers pairs of documents simultaneously in the loss function. The goal is to minimize the number of incorrectly ordered pairs.

Listwise

The model considers the entire list of documents in the loss function. The goal is to find the optimal ordering of the entire list.

Training an LTR Model

The training data for an LTR model typically consists of a list of items, and each item has a set of features and a label (or ground truth). The features might include information about the item itself, and the label typically represents its relevance or importance. For instance, in our recommender systems, we have item features, and in the training dataset, the labels will show if the item is relevant to the user. Additionally, LTR models sometimes make use of the query or user features.

The training process is about learning a ranking function by using these features and labels. These ranking functions are then applied to the retrieved items before serving.

Let's see some examples of how these models are trained.

Classification for Ranking

One way to pose the ranking problem is as a multilabel task. Every item appearing in the training set that is associated to the user is a positive example, while those outside would be negative. This is, in effect, a multilabel approach at the scale of the set of items. The network could have an architecture with each item's features as input nodes, and then some user features as well. The output nodes would be in correspondence with the items you wish to label.

With a linear model, if X is the item vector and Y is the output, we learn W, where $sigmoid(WX) = 1$ if X is an item in the positive set; otherwise, $sigmoid(WX) = 0$. This corresponds to the binary cross-entropy loss in Optax.

Unfortunately, the relative ordering of items isn't taken into account in this setup, so this loss function consisting of sigmoid activation functions for each item won't optimize ranking metrics very well. Effectively, this ranking is merely a downstream *relevance model* that only helps to filter those options retrieved in a previous step.

Another problem with this approach is that we have labeled everything outside of the training set to be negative, but the user might never have seen a new item that could be relevant to a query—so it would be incorrect to label this new item as a negative when it is simply unobserved.

You may have realized that the ranking needs to consider the relative positions in the list. Let's consider this next.

Regression for Ranking

The most naive way to rank a set of items is simply to regress to the rank of a similar number like NDCG or our other personalization metrics that are rank respective.

In practice, this is achieved by conditioning the set of items against a query. For example, we could pose the problem as regression to the

NDCG, given the query as the context of the ranking. Furthermore, we can supply the query as an embedding context vector to a feed-forward network that is concatenated with the features of the items in the set and regress toward the NDCG value.

The query is needed as a context because a set of item's ordering might be dependent upon the query. Consider, for example, typing into a search bar the query `flowers`. We would then expect a set of items most representative of flowers to be in the top results. This demonstrates that the query is an important consideration of the scoring function.

With a linear model, if $\boldsymbol{X}$ is the item vector and $\boldsymbol{Y}$ is the output, then we learn $\boldsymbol{W}$, where $\boldsymbol{WX}(i) = \boldsymbol{NDCG}(i)$ and $\boldsymbol{NDCG}(i)$ is the NDCG for item $\boldsymbol{i}$. Regression can be learned using the L2 loss in Optax.

Ultimately, this approach is about attempting to learn the underlying features of items that lead to higher-rank scores in your personalization metric. Unfortunately, this also fails to explicitly consider the relative ordering of items. This is a pretty serious limitation, which we'll consider shortly.

Another consideration: what do we do for items that aren't ranked outside of the top-*k* training items? The rank we would assign them would be essentially random, as we do not know what number to assign them. Therefore, this method needs improvement, which we'll explore in the next section.

Classification and Regression for Ranking

Suppose we have a web page such as an online bookstore, and users have to browse through and click items in order to purchase them. For such a funnel, we could break the ranking into two parts. The first model could predict the probability of a click on an item, given a set of items on display. The second model could be conditioned on a click-through and could be a regression model estimating the purchase price of the item.

Then, a full ranking model could be the product of two models. The first one computes the probability of clicking through an item, given a set of competing items. And the second one computes the expected value of a purchase, given that it had been clicked. Notice that the first and second model could have different features, depending on the stage of the funnel a user is in. The first model has access to features of competing items, while the second model might take into account shipping costs and discounts applied that might change the value of an item. Thus, in this setting, it would be advantageous to model both stages of the funnel with different models so as to make use of the most amount of information present at each stage of the funnel.

WARP

One possible way to generate a ranking loss stochastically is introduced in "WSABIE: Scaling Up to Large Vocabulary Image Annotation" by Jason Weston et al. The loss is called *weighted approximate rank pairwise* (WARP). In this scheme, the loss function is broken into what looks like a pairwise loss. More precisely, if a higher-ranked item doesn't have a score that is greater than the margin (which is arbitrarily picked to be 1) for a lower-rank item, we apply the *hinge loss* to the pair of items. This looks like the following:

$$max(0, 1 - score(pos) + score(neg))$$

With a linear model, if $\boldsymbol{X}_{\boldsymbol{p}}\boldsymbol{os}$ is the positive item vector, and $\boldsymbol{X}_{\boldsymbol{n}}\boldsymbol{eg}$ is the negative item vector, then we learn $\boldsymbol{W}$, where $\boldsymbol{W}\boldsymbol{X}_{\boldsymbol{p}}\boldsymbol{os} - \boldsymbol{W}\boldsymbol{X}_{\boldsymbol{n}}\boldsymbol{eg} > 1$. The loss for this is hinge loss, where the predictor output is $\boldsymbol{W}\boldsymbol{X}_{\boldsymbol{p}}\boldsymbol{os} - \boldsymbol{W}\boldsymbol{X}_{\boldsymbol{n}}\boldsymbol{eg}$ and the target is 1.

However, to compensate for the fact that an unobserved item might not be a true negative, just something unobserved, we count the number of times we had to sample from the negative set to find something that violates the ordering of the chosen pair. That is, we count the number of times we had to look for something where:

$$score(neg) > score(pos) - 1$$

We then construct a monotonically decreasing function of the number of times we sample the universe of items (less the positives) for a violating negative and look up the weight for this number and multiply the loss with it. If it's very hard to find a violating negative, the gradient should therefore be lower because either we are close to a good solution already or the item was never seen before, so we should not be so confident as to assign it a low score just because it was never shown to the user as a result for a query.

Note that WARP loss was developed when CPUs were the dominant form of computation to train ML models. As such, an approximation to ranking was used to obtain the rank of a negative item. The *approximate rank* is defined as the number of samples with replacement in the universe of items (less the positive example) before we find a negative item whose score is larger than the positive by an arbitrary constant, called a *margin*, of 1.0.

To construct the WARP weight for the pairwise loss, we need a function to go from the approximate rank of the negative item to the WARP weight. A relatively simple bit of code to compute this is as follows:

```
import numpy as np

def get_warp_weights(n: int) -> np.ndarray:
  """Returns N weights to convert a rank to a loss weight."""

  # The alphas are defined as values that are monotonically decreasing.
  # We take the reciprocal of the natural numbers for the alphas.
  rank = np.arange(1.0, n + 1, 1)
  alpha = 1.0 / rank
  weights = alpha

  # This is the L in the paper, defined as the sum of all previous alphas.
  for i in range(1, n):
    weights[i] = weights[i] + weights[i -1]

  # Divide by the rank.
  weights = weights / rank
  return weights
```

```
print(get_warp_weights(5))
[1.         0.75       0.61111111 0.52083333 0.45666667]
```

As you can see, if we find a negative immediately, the WARP weight is 1.0, but if it is very difficult to find a negative that violates the margin, the WARP weight will be small.

This loss function is approximately optimizing precision@*k*, and thus a good step toward improving rank estimates in the retrieved set. Even better, WARP is computationally efficient via sampling and thus more memory efficient.

k-order Statistic

Is there a way to improve upon the WARP loss and straight-up pairwise hinge loss? Turns out there are a whole spectrum of ways. In "Learning to Rank Recommendations with the k-order Statistic Loss", Jason Weston et al. (including one of this book's coauthors) show how this can be done by exploring the variants of losses between hinge loss and WARP loss. The authors of the paper conducted experiments on various corpora and show how the trade-off between optimizing for a single pairwise versus selecting a harder negative like WARP affects metrics including mean rank and precision and recall at *k*.

The key generalization is that instead of a single positive item considered during the gradient step, the model uses all of them.

Recall again that picking a random positive and a random negative pair optimizes for the ROC, or AUC. This isn't great for ranking because it doesn't optimize for the top of the list. WARP loss, on the other hand, optimizes for the top of the ranking list for a single positive item but does not specify how to pick the positive item.

Several alternate strategies can be used for ordering the top of the list, including optimizing for mean maximum rank, which tries to group the positive items such that the lowest-scoring positive item is as near the top of the list as possible. To allow this ordering, we provide a probability dis-

tribution function over how we pick the positive sample. If the probability is skewed toward the top of the positive item list, we get a loss more like WARP loss. If the probability is uniform, we get AUC loss. If the probability is skewed toward the end of the positive item list, we then optimize for the worst case, like mean maximum rank. The NumPy function `np.random.choice` provides a mechanism from sampling from a distribution P.

We have one more optimization to consider: K, the number of positive samples to use to construct the positive set. If $K = 1$, we pick only a positive random item from the positive set; otherwise, we construct the positive set, order the samples by score, and sample from the positive list of size K by using the probability distribution P. This optimization made sense in the era of CPUs when compute was expensive but might not make that much sense these days in the era of GPUs and TPUs, which we will talk about in the following warning.

STOCHASTIC LOSSES AND GPUS

A word of caution about the preceding stochastic losses. They were developed for an earlier era of CPUs when it was cheap and easy to sample and exit if a negative sample was found. These days, with modern GPUs, making branching decisions like this is harder because all the threads on the GPU core have to run the same code over different data in parallel. That usually means both sides of a branch are taken in a batch, so less computational savings occur from these early exits. Consequently, branching code that approximates stochastic losses like WARP and *k*-order statistic loss appear less efficient.

What are we to do? We will show in Chapter 13 how to approximate these losses in code. Long story short, because of the way vector processors like GPUs tend to work by processing lots of data in parallel uniformly, we have to find a GPU-friendly way to compute these losses. In the next chapter, we approximate the negative sampling by generating a large batch of negatives and either scoring them all lower than the negative or looking for the most egregious violating negative or both together as a blend of loss functions.

BM25

While much of this book is targeted at recommending items to users, search ranking is a close sister study. In the space of information retrieval, or search ranking for documents, *best matching 25* (BM25) is an essential tool.

BM25 is an algorithm used in information-retrieval systems to rank documents based on their relevance to a given query. This relevance is determined by considering factors like TF-IDF. It's a bag-of-words retrieval function that ranks a set of documents based on the query terms appearing in each document. It's also a part of the probabilistic relevance framework and is derived from the probabilistic retrieval model.

The BM25 ranking function calculates a score for each document based on the query. The document with the highest score is considered the most relevant to the query.

Here is a simplified version of the BM25 formula:

$$\text{score}\,(D,Q) = \sum_{i=1}^{n} \text{IDF}\,(q_i) * \frac{f\,(q_i,D) * (k1+1)}{f\,(q_i,D) + k1 * \left(1 - b + b * \frac{|D|}{\text{avgdl}}\right)}$$

The elements of this formula are as follows:

- D represents a document.
- Q is the query that consists of words $q_1, q_2, \ldots, q_n\}$.
- $f(qi, D)$ is the frequency of query term q_i in document D.
- $|D|$ is the length of (the number of words in) the document D.
- avg_dl is the average document length in the collection.
- k_1 and b are hyperparameters. k_1 is a positive tuning parameter that calibrates the document term frequency scaling. b is a parameter that determines the scaling by document length: $b = 1$ corresponds to fully scaling the term weight by the document length, while $b = 0$ corresponds to no length normalization.
- $IDF(q_i)$ is the inverse document frequency of query term q_i, which measures the amount of information the word provides (whether it's

common or rare across all documents). BM25 applies a variant of IDF that can be computed as follows:

$$\text{IDF}\left(q_i\right) = \log\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}\right)$$

Here, N is the total number of documents in the collection, and $n(q_i)$ is the number of documents containing q_i.

Simply, BM25 combines both term frequency (how often a term appears in a document) and inverse document frequency (how much unique information a term provides) to calculate the relevance score. It also introduces the concept of document length normalization, penalizing too-long documents and preventing them from dominating shorter ones, which is a common issue in simple TF-IDF models. The free parameters k_1 and b allow the model to be tuned based on the specific characteristics of the document set.

In practice, BM25 provides a robust baseline for most information-retrieval tasks, including ad hoc keyword search and document similarity. BM25 is used in many open source search engines, such as Lucene and Elasticsearch, and is the de facto standard for what is often called *full-text search*.

So how might we integrate BM25 into the problems we discuss in this book? The output from BM25 is a list of documents ranked by relevance to the given query, and then LTR comes into play. You can use the BM25 score as one of the features in an LTR model, along with other features that you believe might influence the relevance of a document to a query.

The general steps to combine BM25 with LTR for ranking are as follows:

1. *Retrieve a list of candidate documents*. Given a query, use BM25 to retrieve a list of candidate documents.
2. *Compute features for each document*. Compute the BM25 score as one of the features, along with other potential features. This could include various document-specific features, query-document match features, user interaction features, etc.

3. *Train/evaluate the LTR model*. Use these feature vectors and their corresponding labels (relevance judgments) to train your LTR model. Or, if you already have a trained model, use it to evaluate and rank the retrieved documents.
4. *Rank*. The LTR model generates a score for each document. Rank the documents based on these scores.

This combination of retrieval (with BM25) and ranking (with LTR) allows you to first narrow the potential candidate documents from a possibly very large collection (where BM25 shines) and then fine-tune the ranking of these candidates with a model that can consider more complex features and interactions (where LTR shines).

It is worth mentioning that the BM25 score can provide a strong baseline in text document retrieval, and depending on the complexity of the problem and the amount of training data you have, LTR may or may not provide significant improvements.

Multimodal Retrieval

Let's take another look at this retrieval method, as we can find some powerful leverage. Think back to Chapter 8: we built a co-occurrence model, which illustrated how articles referenced jointly in other articles share meaning and mutual relevance. But how would you integrate search into this?

You may think, "Oh, I can search the names of the articles." But that doesn't quite utilize our co-occurrence model; it underleverages that joint meaning we discovered. A classic approach may be to use something like BM25 on article titles or articles. More modern approaches may do a vector embedding of the query and article titles (using something like BERT or other transformer models). However, neither of these really capture both sides of what we're looking for.

Consider instead the following approach:

1. Search with the initial query via BM25 to get an initial set of "anchors."
2. Search with each anchor as a query via your latent model(s).
3. Train an LTR model to aggregate and rank the union of the searches.

Now we're using a true multimodal retrieval, leveraging multiple latent spaces! One additional highlight in this approach is that queries are often out of distribution from documents with respect to encoder-based latent spaces. This means that when you type `Who's the leader of Mozambique?`, this question looks fairly dissimilar to the article title (Mozambique) or the relevant sentence as of summer 2023 ("The new government under President Samora Machel established a one-party state based on Marxist principles.")

When the embeddings are not text at all, this method becomes even more powerful: consider typing text to search for an item of clothing and hoping to see an entire outfit that goes with it.

Summary

Getting things in the right order is an important aspect of recommendation systems. By now, you know that ordering is not the whole story, but it's an essential step in the pipeline. We've collected our items and put them in the right order, and all that's left to do is send them off to the user.

We started with the most fundamental concept, learning to rank, and compared it with some traditional methods. We then got a big upgrade with WARP and WSABIE. That led us to the *k*-order statistic, which involves utilizing more careful probabilistic sampling. We finally wrapped up with BM25 as a powerful baseline in text settings.

Before we conquer serving, let's put these pieces together. In the next chapter, we're going to turn up the volume and build some playlists. This will be the most intensive chapter yet, so go grab a beverage and a stretch. We've got some work to do.